

can elevate our permission using *sudo* to get the result.

```
> sudo netstat -nlt --program
[sudo] password for sibi:
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      1110/sshd: /nix/sto
tcp6       0      0 :::22                   :::*                     LISTEN      1110/sshd: /nix/sto
```

You can see that it's the SSH daemon that has opened the port on 22. You can even remove the *-n* option so that it prints the name of the port.

```
> sudo netstat -lt --program
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:ssh             0.0.0.0:*               LISTEN      1110/sshd: /nix/sto
tcp6       0      0 [::]:ssh                 [::]:*                  LISTEN      1110/sshd: /nix/sto
```

The 0.0.0.0 at the local address indicates that it is listening on all the network interfaces (your network card, 127.0.0.1, etc). If this was 127.0.0.1, it would mean that connections are from the machine itself, and not from the network or Internet, are being listened to. If the local network IP (192.168.3.107 in this case) is displayed, it means that connections will be allowed from the local network as well.

In fact, you can make a program listen to your local network and use *netstat* to confirm it:

```
> python3 -m http.server --bind 192.168.3.107
Serving HTTP on 192.168.3.107 port 8000
(http://192.168.3.107:8000/) ...
Verification can be done on another terminal session:

> sudo netstat -lnt --program
```

Active Internet connections (only servers)

```
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      1110/sshd: /nix/sto
tcp        0      0 192.168.3.107:8000      0.0.0.0:*               LISTEN      21410/python3
tcp6       0      0 :::22                   :::*                     LISTEN      1110/sshd: /nix/sto
```

Looking further

While *netstat* is obsolete, learning it is still valuable because of the ubiquitous nature of this tool. Having a good grasp of how to read its output will be quite handy when networking issues have to be debugged. With various functionalities, *netstat* has a wide scope and can be used to view even the routing table (which is usually done by IP r):

```
> netstat -rn
Kernel IP routing table
Destination Gateway Genmask Flags MSS Window irtt Iface
0.0.0.0 192.168.3.1 0.0.0.0 UG 0 0 0 eno1
0.0.0.0 192.168.0.1 0.0.0.0 UG 0 0 0 wlp0s20f3
0.0.0.0 192.168.0.1 0.0.0.0 UG 0 0 0 wlp0s20f3
172.17.0.0 0.0.0.0 255.255.0.0 U 0 0 0 docker0
192.168.0.0 0.0.0.0 255.255.255.0 U 0 0 0 wlp0s20f3
192.168.0.0 0.0.0.0 255.255.255.0 U 0 0 0 wlp0s20f3
192.168.3.0 0.0.0.0 255.255.255.0 U 0 0 0 eno1
```

The manual page is given under 'References' to understand the other functionalities that *netstat* provides. If you are comfortable reading its output, you will be better equipped to debug your networking problems. **END** 🐼

Reference

Manual page: <https://linux.die.net/man/8/netstat>

By: Sibi Prabakaran

The author is a senior engineer working at FPCComplete.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU